Structs, Méthodes et Composition

Objectifs

1. Optimiser l'empreinte RAM d'une `struct` en réorganisant ses champs pour éliminer le bourrage mémoire (*Struct Padding*) à l'aide du package `unsafe`.
2. Choisir et implémenter des méthodes avec *Value Receiver* ou *Pointer Receiver*, et sécuriser une méthode appelée sur un pointeur `nil`.
3. Architecturer des modèles métiers complexes par composition (*Struct Embedding*) avec promotion et masquage (*Shadowing*) de méthodes.

Exercice 1 : Optimisation mémoire et *Struct Padding* (IoT haute densité)

Vous développez le serveur central d'un opérateur IoT qui conserve en RAM l'état de 50 millions de capteurs connectés. Un développeur a écrit la structure suivante sans se soucier de l'alignement mémoire 64 bits :

```Go
type CapteurNonOptimise struct {
	EstActif      bool    // 1 octet
	ID            uint64  // 8 octets
	BatterieFaible bool   // 1 octet
	ValeurMesuree float64 // 8 octets
	CodeZone      uint8   // 1 octet
	PortReseau    uint16  // 2 octets
	EnAlerte      bool    // 1 octet
}
```

1. À l'aide du package `"unsafe"`, affichez la taille totale en octets occupée par une instance de `CapteurNonOptimise` (`unsafe.Sizeof(...)`) ainsi que l'offset mémoire exact de chacun de ses champs (`unsafe.Offsetof(...)`).
2. **Analyse du gaspillage :**
   - La somme arithmétique des champs est de $1 + 8 + 1 + 8 + 1 + 2 + 1 = 22$ octets. Pourquoi la structure occupe-t-elle **40 octets** en RAM ?
   - Pour 50 millions de capteurs en mémoire, combien de mégaoctets sont gaspillés uniquement en octets de bourrage (*padding*) ?
3. **Optimisation :**

- Créez une seconde structure `CapteurOptimise` contenant exactement les mêmes champs (mêmes noms et mêmes types), mais réordonnés de manière optimale.
- Vérifiez avec `unsafe.Sizeof` que `CapteurOptimise` ne pèse plus que **24 octets** (soit une économie de 40 % de RAM, ou **800 Mo** gagnés sur 50 millions d'instances !).

Exercice 2 : *Value Receiver*, *Pointer Receiver* et robustesse au `nil`

Vous devez implémenter le moteur de gestion d'un portefeuille financier (`CompteBancaire`) et démontrer pourquoi un mauvais choix de *Receiver* entraîne des bugs silencieux.

1. Définissez une structure `CompteBancaire` contenant :
   - `Titulaire` (`string`)
   - `Solde` (`float64`)
   - `Verrouille` (`bool`)
2. Créez une fonction constructeur idiomatique `NewCompteBancaire(titulaire string, soldeInitial float64) *CompteBancaire` qui initialise et retourne un pointeur vers le compte (si `soldeInitial < 0`, forcez-le à `0`).
3. **Le piège du *Value Receiver* :**
   - Écrivez une méthode `CrediterBug(montant float64)` utilisant un **Value Receiver** `(c CompteBancaire)` qui ajoute `montant` à `c.Solde`.
   - Écrivez une méthode `Crediter(montant float64)` utilisant un **Pointer Receiver** `(c *CompteBancaire)` qui ajoute `montant` à `c.Solde` (uniquement si `!c.Verrouille` et `montant > 0`).
   - Dans `main()`, appelez successivement `CrediterBug(500)` puis `Crediter(500)` sur votre compte et affichez le solde après chaque appel.
4. **Sécurisation d'un *Nil Receiver* :**
   - Écrivez une méthode `AfficherResume() string` avec un **Pointer Receiver** `(c *CompteBancaire)`.
   - Si cette méthode est appelée sur un pointeur `nil` (ex: `var compteInexistant *CompteBancaire; fmt.Println(compteInexistant.AfficherResume())`), elle ne doit **pas** provoquer de `panic` : interceptez le cas `if c == nil` pour retourner proprement `"<Compte invalide ou nil>"`.

Exercice 3 : Composition par *Struct Embedding* et *Shadowing*

Dans une architecture de microservices, vous souhaitez partager des briques communes (audit, métriques réseau) entre différents types de composants sans utiliser d'héritage POO.

1. **Brique de base n°1 (`JournalAudit`) :**
   - Créez une structure `JournalAudit` contenant une slice `Historique []string`.
   - Attachez-lui une méthode `Enregistrer(evenement string)` (sur *Pointer Receiver*) qui ajoute l'événement dans `Historique`, et une méthode `Rapport() string` qui retourne le nombre d'événements enregistrés.
2. **Brique de base n°2 (`ConfigReseau`) :**
   - Créez une structure `ConfigReseau` avec les champs `IP string` et `Port int`.
   - Attachez-lui une méthode `AdresseComplete() string` qui retourne `"<IP>:<Port>"` (utilisez `fmt.Sprintf`).
3. **Composition dans `WorkerPaiement` :**
   - Créez une structure `WorkerPaiement` qui **imbrique anonymement** (*Struct Embedding*) `JournalAudit` et `ConfigReseau`, et qui ajoute un champ propre `IDWorker string`.
4. **Promotion automatique :**
   - Dans `main()`, instanciez un `WorkerPaiement`.
   - Sans jamais écrire `.JournalAudit` ni `.ConfigReseau`, accédez directement à `worker.IP`, appelez `worker.AdresseComplete()` et enregistrez deux actions via `worker.Enregistrer(...)`.
5. **Masquage de méthode (*Shadowing*) :**
   - Implémentez une méthode `Rapport() string` directement sur `WorkerPaiement`.
   - Cette méthode doit retourner une chaîne combinant l'`IDWorker`, l'`AdresseComplete()` promue, **ainsi que** le résultat de la méthode `Rapport()` originale de la brique `JournalAudit` imbriquée.
   - Vérifiez dans `main()` que `worker.Rapport()` appelle bien la version surchargée de `WorkerPaiement`.